

View-oriented Conversation Compiler for Agent Trace Analysis

Lvmin Zhang and Maneesh Agrawala
Stanford University

Agent traces carry increasing analytical value in the era of context learning and harness-driven agentic cognition, yet most prior work treats conversation format as a trivial engineering detail. Modern agent conversations, however, contain deeply structured content—nested tool calls and results, chain-of-thought reasoning blocks, sub-agent invocations, context-window compaction boundaries, and harness-injected system directives—whose complexity far exceeds that of simple user–assistant exchanges. Feeding such traces to a reflector or other analytical mechanism in plain text, JSON, YAML, or via grep can materially degrade analysis quality. This paper presents VCC (View-oriented Conversation Compiler), a compiler (lex $\rightarrow$ parse $\rightarrow$ IR $\rightarrow$ lower $\rightarrow$ emit) that transforms raw agent JSONL logs into a family of structured views: a *full view* (lossless transcript serving as the canonical line-number coordinate system), a *user-interface (UI) view* (reconstructing the interaction as the user actually perceived it), and an *adaptive view* (a structure-preserving projection governed by a relevance predicate). In a context-learning experiment on AppWorld, replacing only the reflector’s input format—from raw JSONL to VCC-compiled views—leads to higher pass rates across all three model configurations tested, while cutting reflector token consumption by half to two-thirds and producing more concise learned memory. These results suggest that message format functions as infrastructure for context learning, not as an incidental implementation choice.

1 Introduction

Context learning research has devoted considerable attention to upper-level architectural choices: role decomposition among generators, reflectors, and curators [58]; hierarchical memory organization and retrieval [29, 31]; tool use [32]; and experience extraction and generalization from agent trajectories [8, 39, 35]. The format of the messages themselves—how an agent trace is represented and presented to an LLM for analysis—has largely been treated as a trivial engineering detail, with plain text, JSON, YAML, and grep serving as default conventions.

For simple user–assistant exchanges, these conventions may suffice. Contemporary traces, however, carry a different kind of complexity. A single session from a coding agent such as Claude Code or OpenHands [44] can produce a JSONL log exceeding ten thousand lines, in which tool-call parameters, tool-result bodies, chain-of-thought reasoning blocks [46], sub-agent invocations, context-compaction boundaries, base64-encoded media, and harness-injected system directives are interleaved. Tool-result bodies alone can account for the majority of the token budget, burying the actual decision points beneath volumes of file contents and terminal output. Prior empirical work supports the view that format is not neutral: identical content presented in different formats (plain text, Markdown, JSON, YAML) can cause LLM performance to vary by up to 40 percentage points on certain tasks [12], and 40–60% of tokens in agent trajectories have been identified as useless, redundant, or expired information whose removal does not degrade task performance [48]. The bottleneck, in other words, is not a lack of capable learning methods but the quality of their input.

This paper presents VCC (View-oriented Conversation Compiler), a compiler that transforms raw agent JSONL logs into structured views. VCC follows a full compiler pipeline—lexing, parsing to a typed intermediate representation (IR), line-number assignment, and view-specific lowering—to produce three views. The *full view* emits every IR node verbatim, serving as the lossless canonical transcript whose line numbers define the coordinate system shared by all views. The *UI view* reconstructs the interaction as the user perceived it during the original session: internal content (chain-of-thought blocks, tool-call bodies, tool results, system messages) is elided, each tool invocation is collapsed to a one-line summary annotated with a pointer into the full view, and consecutive assistant turns are merged. The *adaptive view* projects the conversation through a relevance predicate ρ (currently instantiated as regex matching, though the framework accommodates BM25, embedding-based, or LLM-based predicates), retaining the structural skeleton—section headers, turn boundaries, block delimiters—while emitting only matching content blocks, each annotated with its semantic role and a line-range pointer. A key design invariant is that all line-number references across views resolve to the same coordinate system, and this consistency is guaranteed by construction: line numbers are assigned exactly once before any view-specific lowering, and lowering passes may only select, truncate, or annotate content—never reorder or renumber it.

We point out two problem-modeling concepts in VCC. First, VCC’s multiple views are not a multi-level memory hierarchy in the sense of MemGPT [29], A-MEM [49], or RAPTOR [31]. Those systems maintain static, precomputed artifacts (summaries, embeddings, knowledge-graph nodes) connected by persistent edges; VCC’s views are dynamic, query-time projections of a single flat IR, with no precomputed summaries and no stored state. More precisely, multi-level memory performs *abstraction*—generating new content not present in the source—while VCC performs *projection*—selecting and annotating existing content with exact coordinates for recovering the remainder (Section 2.1). Second, the adaptive view differs from grep search. Flat grep returns isolated matching lines stripped of their conversational identity; the adaptive view preserves block-level semantic role annotations and range pointers, enabling an agent to distinguish whether a match appeared in a user utterance, a chain-of-thought block, a tool-call input, or a tool result (Section 2.1).

VCC is not a single-layer format conversion (JSON $\rightarrow$ Markdown). The compiler executes numerous trace-specific transformations: tool-call parameters are compiled from escaped JSON into readable YAML with block-scalar multi-line strings; harness-injected XML markup (`<system-reminder>`, `<ide_opened_file>`, `<command-message>`, among others) is identified and removed, and user turns consisting entirely of such markup are hidden; assistant message fragments split across JSONL records by context compaction (sharing the same message ID) are reassembled into single sections. The full catalog of transformations appears in Section 2.3.

To evaluate whether trace format materially affects learning outcomes, we conduct a context-learning experiment on AppWorld [41]. A generator agent executes tasks; a reflector agent analyzes the resulting trace and edits a shared memory file to accumulate rules. The sole experimental variable is the reflector’s input format: VCC-compiled views versus raw JSONL. Across three model configurations (Opus, Sonnet, Haiku) and two test splits (168 and 416 tasks), VCC-trained memory leads to higher pass rates, while cutting reflector token consumption by half to two-thirds and producing more concise memory files. The details are presented in Section 3.2.

2 VCC

2.1 Three Views

An agentic conversation trace is a temporally ordered sequence of heterogeneous records: user utterances, assistant responses (interleaving natural language, chain-of-thought reasoning, and tool invocations), tool results, and system directives. VCC formalizes this trace as a structured document and defines three *views*—projections of a shared intermediate representation that vary in resolution and coverage while maintaining referential consistency (Figure 1).

Full view. The full view is the identity projection. Let $\mathcal{I} = (n_1, n_2, \dots, n_N)$ denote the IR node sequence after parsing and line assignment. The full view emits every node verbatim:

$$V_{\text{full}} = \bigoplus_{i=1}^N \text{emit}(n_i), \quad (1)$$

where $\oplus$ denotes concatenation with structurally determined blank-line insertion. This rendering is bijective: the source can be reconstructed (up to serialization whitespace) from V_{full} . The full view serves as the canonical reference surface—all line numbers in other views resolve against it.

User-Interface (UI) view. The UI view reconstructs the perceptual experience of the human user during the original interactive session. In a terminal-based coding agent, the user seldom sees raw tool inputs, tool outputs, or chain-of-thought reasoning; the user observes assistant text, one-line tool-call summaries, and the conversational turn structure. The UI view applies four transformations to the IR. First, *internal content is elided*: tool-call bodies, tool-result bodies, thinking blocks, and system messages are suppressed. Second, *summary substitution* collapses each tool call to a single-line summary (e.g. * Read "src/config.py" (file.txt:19-21,24-34)) annotated with line-range pointers into V_{full} . Third, *consecutive-turn merging* combines adjacent assistant sections with no intervening user turn, reflecting the user’s perception of a single continuous response. Fourth, *noise filtering* strips markup injected by the harness (IDE selection contexts, system reminders, hook outputs), as these are not part of the user’s intentional input.

Adaptive view. The adaptive view generalizes content-based retrieval by combining a relevance predicate with structure-preserving projection. Given a predicate $\rho : \text{line} \rightarrow \{0, 1\}$, the adaptive view retains the structural skeleton of the conversation—section headers, turn boundaries, block delimiters—while projecting each content block through ρ :

$$V_{\text{adapt}}(b) = \begin{cases} \text{match_lines}(b, \rho) & \text{if } \exists l \in [s, e] : \rho(l) = 1 \\ \emptyset & \text{otherwise,} \end{cases} \quad (2)$$

where b spans lines $[s, e]$ in V_{full} , and `match_lines` returns matching lines annotated with their V_{full} line numbers and prefixed by a block-level pointer ($f : \text{Ls-Le}$). Sections with no matching content are elided entirely. The predicate ρ is a parameter: regex matching is the simplest instantiation, but BM25 scoring, embedding-based similarity, or LLM-based relevance judgments slot into the same framework without altering the structural projection logic.

The adaptive view admits two output modalities that stand in a transpose relationship. The *document-oriented* modality preserves the temporal structure of the conversation, organized section by section with non-matching blocks elided. The *index-oriented* modality discards structural scaffolding and emits a flat sequence of matching blocks, each tagged with its semantic role (e.g. `[tool_call]`, `[assistant]`, `[thinking]`). The underlying data is identical; only the iteration order differs—row-major by section versus column-major by match.

Distinction from multi-level memory. Multi-level memory architectures (MemGPT [29], A-MEM [49], Mem0 [4], RAPTOR [31]) maintain static, precomputed artifacts—summaries, embeddings, knowledge-graph nodes—connected by persistent edges. Querying such a structure means traversing a pre-existing topology; the levels exist independently of any particular query. VCC’s views, by contrast, are dynamic, query-time projections of a single flat IR. No precomputed summary exists; no persistent graph is maintained; no embedding is stored. Different predicates produce different views by recomputing the projection from scratch. More fundamentally, multi-level memory performs *abstraction*: generating new content (summaries, embeddings) not present in the source, introducing lossy compression whose faithfulness must be evaluated empirically. VCC performs *projection*: selecting and arranging existing content from the source, annotated with exact coordinates for recovering the unselected remainder. The system is lossless by construction—not in the sense that every view contains all information, but in the sense that every view contains a verifiable path back to the full information.

Distinction from grep. A flat grep over a conversation trace returns a set of isolated matching lines. Each line is stripped of its conversational identity: the reader cannot determine whether a match occurred in a user utterance, an assistant’s chain-of-thought, a tool-call input, or a tool result. A match tagged `[tool_call]` Edit (an intention) carries different implications than a match tagged `[tool_result]` Read (an observation)—grep collapses intention, observation, and reasoning into a uniform bag of lines. The adaptive view preserves block-level role annotations and range pointers, enabling semantic distinctions that flat search cannot express. Additionally, the block-level pointers carry range information, not just point locations: a match at line 50 within a block spanning lines 46–56 tells the reader that ten lines of surrounding context exist at a known, dereferenceable location.



Figure 1: The same agent trace rendered under three views. The full view (left) emits every IR node with line numbers. The UI view (center) collapses tool calls to one-line summaries with line-range pointers (purple arrows). The adaptive view (right, grep “dog”) retains only matching blocks with role annotations and range pointers (blue arrows). Highlighted lines show matching content; all pointers resolve to the full view’s line-number coordinate system.

A more sophisticated alternative—decomposing the trace into per-message files and using filesystem grep (or structured grep)—fails on a specific property: context contiguity upon dereference. In the single-file full view, a pointer dereference yields immediately adjacent surrounding context in $O(1)$; in a per-file decomposition, recovering the same situated understanding requires $O(k)$ reads across separate files plus manual reassembly of the temporal sequence.

2.2 Compiler Pipeline and Line-Number Consistency

VCC follows a full compiler pipeline with four stages: (i) *lexing* tokenizes the JSONL source into typed records, one per source record; records carrying no conversational content (queue-operation, file-history-snapshot, progress, api_error, among others) are filtered at this stage; (ii) *parsing* lowers records to a typed intermediate representation where each node carries a semantic type (user, assistant, tool_call, tool_result, thinking, etc.), its content lines, and structural metadata (section index, block index); (iii) *line assignment* performs a single sequential pass that assigns monotonically increasing line numbers to all IR nodes—this assignment is deterministic and occurs exactly once, before any view-specific lowering; (iv) *view lowering* projects the shared IR into each view: the full view emits all nodes; the UI view and adaptive view apply their respective transformations. No lowering pass modifies the line-number assignment—lowering may only select, truncate, or annotate content, never reorder or renumber it.

Because all three views are projections of a single IR with a single line-number assignment, pointer consistency across views is a structural invariant of the pipeline rather than a property that must be validated post hoc. This is analogous to how a compiler’s static single-assignment form guarantees use-def consistency by construction rather than by testing.

2.3 Compiler Transformations

As with any compiler, the lexer and parser are specific to the source language—in this case, the Claude Code JSONL “language”. The IR, line-assignment, and view-lowering stages are language-independent; supporting a different agent runtime (e.g. Codex, OpenClaw) requires writing a new lexer and parser while reusing the rest of the pipeline.

We briefly describe the Claude Code lexer and parser to illustrate the scope. Tool-call parameters are escaped JSON strings (e.g. `{"file_path": "src/pets.py", "content": "class Pet:\n\ndef __init__..."}`); the lexer compiles them to YAML with block scalars (|) for multi-line values. Read-tool results carry a `digits→content` prefix that the parser strips to recover the original source. Harness-injected XML markup (`<system-reminder>`, `<ide_opened_file>`, `<command-message>`, among others) is removed, and user turns consisting entirely of such markup are elided. Internal tool calls (TodoWrite, ToolSearch), ANSI escape codes, and control characters are stripped. When context compaction splits an assistant message across two JSONL records with the same message.id, the parser reassembles them into one section. Record types with no conversational content (queue-operation, file-history-snapshot, progress, api_error, among others) are discarded during lexing. Base64-encoded inline images are decoded to separate files and replaced by placeholders.

2.4 Agent Workflow

The three views enable a progressive-disclosure workflow. An agent (or human analyst) begins with the UI view to scan the overall session trajectory—the sequence of user requests, assistant actions, and outcomes—without wading through tool I/O or chain-of-thought internals. Targeted searches via the adaptive view then locate specific content blocks of interest, each annotated with its semantic role and a line-range pointer. Following a pointer into the full view recovers the complete, verbatim content of that block and its immediate surroundings:

$$V_{\text{ui}} \xrightarrow{\text{pointer}} V_{\text{full}}[s:e], \quad V_{\text{adapt}} \xrightarrow{\text{pointer}} V_{\text{full}}[s:e]. \quad (3)$$

In the experiment described in Section 3, the reflector reads the UI view to establish an overall understanding of the session, then follows pointers to inspect the full content of specific tool calls and results.

3 Experiment & Results

3.1 Setup

Benchmark. We use AppWorld [41], which provides a realistic API-interaction environment for evaluating autonomous agents. The benchmark offers three splits: *train* (90 tasks) for learning, *test_normal* (168 tasks) and *test_challenge* (416 tasks) for evaluation. Tasks span three difficulty levels (d1, d2, d3).

Learning procedure. We adopt an epoch-based context-learning protocol. In each epoch, every task in the train split is executed once by a *generator* agent, then evaluated by the AppWorld harness. A *reflector* agent reads the evaluation result to determine success or failure, analyzes the generator’s execution trace, and edits a shared memory file (`MEMORY.md`) to add rules. For failed tasks, the reflector writes [CRITICAL] rules identifying root causes and preventive measures; for successful tasks, the reflector writes rules flagging risky or suboptimal patterns. The process runs for two epochs (90 tasks $\times$ 2 = 180 analyses per condition), iteratively refining the rule set. Training runs 32 tasks in parallel; after each batch, workers’ memory edits are merged via an LLM-based diff-and-merge step. The learned `MEMORY.md` is then evaluated on the test splits in a single pass with no further memory updates.

Experimental variable. The sole variable is the format of the trace presented to the reflector during analysis: In the *VCC* condition, the raw JSONL is pre-compiled into structured views; the reflector receives the UI view (for session-level orientation) and the full view (for line-level detail), navigating between them via line-number pointers. In the *JSON* condition, the reflector reads the raw JSONL directly, which is the default approach in existing trajectory-learning methods.

Model configurations. Three generator–reflector pairings are tested: Opus+Opus, Sonnet+Sonnet, and Haiku+Sonnet. The original design called for same-model pairing throughout, but Haiku does not have reflector capability (unable to improve pass rates over any candidates in our experiments), and the Haiku reflector was replaced with Sonnet.

Baselines. For each generator model, a baseline evaluation uses only the initial seed memory (no trained rules) on both test splits, establishing the lower bound for context-learning gains.

Metrics. We report AppWorld’s official metrics: *task_goal_completion* (whether the task’s primary objective is met) and *scenario_goal_completion* (a stricter measure requiring all scenario-level constraints—side effects, state consistency, edge cases—to be satisfied in addition to the primary goal), broken down by difficulty level (d1 = easy, d2 = medium, d3 = hard). We also report total reflector token consumption during training (T_{tok}), per-analysis token cost (T/a), and the size of the learned memory file in characters (Mem).

3.2 Results

Table 1 presents the full results for each model configuration.

Pass rate. VCC leads in aggregate *task_goal_completion* across all three model configurations on both test splits: Opus 94.0 vs. 92.9 on *test_normal* and 86.3 vs. 84.7 on *test_challenge*; Sonnet 92.9 vs. 90.5 and 74.3 vs. 71.0; Haiku 75.6 vs. 71.4 and 54.7 vs. 53.5. On the stricter *scenario_goal* metric, for Opus on *test_normal*, VCC reports 91.1 compared with 87.5 for JSON, a difference of 3.6 percentage points.

Token consumption. VCC cuts reflector token consumption by half to two-thirds compared with the JSON baseline: Opus 7.6M vs. 22.1M, Sonnet 8.8M vs. 20.5M, Haiku+Sonnet 12.6M vs. 23.3M. Per-analysis costs follow the same pattern (Opus 43K vs. 124K). The reduction is a direct consequence of the UI view’s compression: the reflector reads a concise session overview rather than navigating thousands of lines of raw JSONL, and follows pointers only to the blocks that require detailed inspection.

Table 1: AppWorld results across three model configurations. All pass rates in %. TN = test_normal (168 tasks), TC = test_challenge (416 tasks). T tok = total reflector tokens during training, T/a = tokens per analysis, Mem = learned memory size (characters). Between JSON and VCC, the higher pass rate and the lower cost are in **bold**.

(a) Claude Code 2.1.81: claude-opus-4-6 (generator) + claude-opus-4-6 (reflector)																			
Method	TN task_goal				TN scenario_goal				TC task_goal				TC scenario_goal				T tok	T/a	Mem
	all	d1	d2	d3	all	d1	d2	d3	all	d1	d2	d3	all	d1	d2	d3			
Baseline	79.2	77.2	75.0	84.1	67.9	68.4	56.2	76.2	75.8	69.4	76.7	77.4	64.0	54.2	66.0	66.1	—	—	
JSON	92.9	96.5	93.8	88.9	87.5	89.5	87.5	85.7	84.7	91.7	84.7	82.0	70.5	83.3	74.0	63.1	22.1M	124K	
VCC	94.0	98.2	95.8	88.9	91.1	94.7	93.8	85.7	86.3	93.1	84.7	85.1	74.1	87.5	70.0	72.3	7.6M	43K	

(b) Claude Code 2.1.81: claude-sonnet-4-6 (generator) + claude-sonnet-4-6 (reflector)																			
Method	TN task_goal				TN scenario_goal				TC task_goal				TC scenario_goal				T tok	T/a	Mem
	all	d1	d2	d3	all	d1	d2	d3	all	d1	d2	d3	all	d1	d2	d3			
Baseline	73.2	77.2	58.3	81.0	67.9	73.7	50.0	76.2	61.4	61.1	59.3	63.1	50.4	45.8	50.0	52.3	—	—	
JSON	90.5	98.2	91.7	82.5	82.1	94.7	81.2	71.4	71.0	86.1	66.7	68.7	60.4	75.0	54.0	60.0	20.5M	115K	
VCC	92.9	96.5	95.8	87.3	89.3	94.7	93.8	81.0	74.3	88.9	68.7	73.3	63.3	79.2	56.0	63.1	8.8M	49K	

(c) Claude Code 2.1.81: claude-haiku-4-5 (generator) + claude-sonnet-4-6 (reflector)																			
Method	TN task_goal				TN scenario_goal				TC task_goal				TC scenario_goal				T tok	T/a	Mem
	all	d1	d2	d3	all	d1	d2	d3	all	d1	d2	d3	all	d1	d2	d3			
Baseline	39.9	49.1	27.1	41.3	25.0	36.8	12.5	23.8	33.8	31.9	34.0	34.4	17.3	16.7	22.0	13.9	—	—	
JSON	71.4	86.0	79.2	52.4	62.5	79.0	75.0	38.1	53.5	69.4	45.3	53.8	36.0	45.8	30.0	36.9	23.3M	131K	
VCC	75.6	87.7	75.0	65.1	62.5	68.4	56.2	61.9	54.7	81.9	47.3	50.3	39.6	66.7	38.0	30.8	12.6M	71K	

Memory size. VCC-trained memory files are consistently smaller: Opus 12.0K vs. 12.6K characters, Sonnet 15.5K vs. 23.4K, Haiku 37.9K vs. 43.3K. Smaller memory correlates with higher pass rates in this experiment, suggesting that VCC enables the reflector to produce more focused rules rather than accumulating verbose or redundant entries.

Weak-model benefit. The Haiku configuration shows the largest absolute gain from context learning (baseline 39.9 → VCC 75.6 on test_normal task_goal, an increase of 35.7 percentage points) and the largest VCC-over-JSON advantage on test_normal task_goal (75.6 vs. 71.4, a difference of 4.2 percentage points). This is likely because a weaker generator produces more error-prone traces that are harder to analyze in raw form, making the structured views more beneficial for the reflector.

JSON behaviors. JSON reports higher scores than VCC in certain per-difficulty cells—for instance, Haiku d2 on test_normal task_goal (79.2 vs. 75.0) and several scenario_goal cells in the Haiku configuration. These cases tend to occur in specific difficulty-split combinations rather than at the aggregate level. One possible explanation is that the JSON reflector, consuming more tokens per analysis, occasionally captures details that the VCC reflector’s pointer-based navigation misses, at the cost of a much larger overall token budget.

Discussion. Higher pass rates, lower token consumption, and smaller memory files are not independent outcomes. A plausible shared cause is that structured views let the reflector focus on decision-relevant content rather than navigating raw data. This produces rules that are both more precise (higher pass rates) and more concise (smaller memory), while the navigation structure itself reduces the volume of text the reflector must process (fewer tokens). The three metrics may be measuring the same underlying variable—the degree to which the reflector’s attention is directed at the right information.

4 Related Work

Format affects LLM performance. He *et al.* [12] show that formatting identical content as plain text, Markdown, JSON, or YAML causes performance to vary by up to 40 percentage points; Tam *et al.* [40] find that structured-output constraints degrade reasoning ability; Sui *et al.* [36] observe that table serialization format changes accuracy by approximately 7 points. On the length axis, Liu *et al.* [24] document a U-shaped attention pattern where mid-context information is systematically underused, and Du *et al.* [7] show that input length alone degrades performance by 13.9–85% even

with perfect retrieval. Within agent trajectories, Xiao *et al.* [48] find that 40–60% of tokens are redundant, and Lindenbauer *et al.* [23] show that simple observation masking matches LLM-based summarization at half the cost. Performance degradation with increasing input length is also documented in [42]. Liu *et al.* [26] show that jointly optimizing prompt content and format outperforms content-only optimization. Yang *et al.* [50] demonstrate that agent-computer interface design affects performance as much as the underlying model. Mei *et al.* [28] survey context engineering as an emerging discipline.

Trajectory learning methods. A growing body of work extracts reusable experience from agent trajectories. ReAct [52] interleaves reasoning traces with actions; Reflexion [34] stores verbal self-reflections in episodic memory; ExpeL [59] contrasts successful and failed trajectories to extract rules. TIM [8] derives strategy and recovery tips from trajectories (+14.3 percentage points on AppWorld); ACE [58] evolves a playbook via generator/reflector/curator roles; Dynamic Cheatsheet [39] maintains a reusable strategy table at test time. ETO [35] learns from contrastive trajectory pairs via DPO; ECHO [13] generates counterfactual success trajectories via hindsight rewriting; RPMS [55] introduces rule–memory conflict arbitration. AutoGuide [11] generates conditional guidelines from offline experience; AutoManual [3] constructs instruction manuals through multi-agent interaction; Agent Workflow Memory [45] induces reusable workflows (+51.1% on WebArena); MACLA [10] distills trajectories into procedural memory with Bayesian reliability tracking. All of these methods consume traces in their raw format. VCC is a preprocessing layer that structures their input.

Structured trace analysis. AgentRx [2] normalizes heterogeneous agent logs to a common IR and synthesizes executable constraints for failure diagnosis (+23.6 percentage points in failure localization). TraceSIR [51] uses a multi-agent framework with an LLM-based importance scorer for trace reduction. The Cognition Agent Trace specification [5] defines a structured JSON format for agent provenance. AgentStepper [15] represents trajectories as structured conversations and provides an interactive debugger with breakpoints and stepwise execution. TRAIL [6] benchmarks LLM trace-debugging ability on 148 human-annotated traces and reports that the best model scores 11%; AgenTracer [57] finds that current reasoning LLMs achieve below 10% accuracy on failure attribution in multi-agent traces. DoVer [27] augments hypothesis-based debugging with active intervention and replay, recovering 18–28% of failed trials. LOCA-bench [56] benchmarks agent performance under controllable context growth. The compiler metaphor has been applied elsewhere in LLM systems: DSPy [19] compiles prompt-pipeline parameters and SGLang [60] compiles LLM programs for efficient execution.

Agent memory and context management. Multi-level memory systems—MemGPT [29], A-MEM [49], Mem0 [4], RAPTOR [31]—organize stored experience through hierarchical or graph-based structures, building on the memory-stream architecture of Park *et al.* [30]; Hu *et al.* [14] survey the broader landscape. Sumers *et al.* [37] propose CoALA, a cognitive architecture organizing agents around modular memory and structured action spaces. Context-compression methods include LLM-Lingua [16] (up to 20x prompt compression), AgentFold [54] and ACON [18] (runtime observation compression), and AgentRM [33] and AgentOCR [9] (learned compression). More recent work gives agents control over their own context: Focus [43] lets the agent decide when to consolidate and prune history, and Context-Folding [38] folds completed subtasks to reduce active context by 10x. Multi-agent frameworks such as AutoGen [47] produce particularly complex traces. Observability platforms (LangSmith [21], Langfuse [22], AgentTrace [1]) instrument running agents for structured telemetry. Memory-pointer approaches [20] for context-window overflow are conceptually related to VCC’s cross-view line-number pointers.

5 Conclusion

VCC compiles raw agent JSONL logs into semantically annotated, line-number-consistent, pointer-navigable structured views. Replacing only the reflector’s input format—from raw JSONL to VCC-compiled views—yields higher pass rates, lower token consumption, and more concise learned memory across three model configurations on AppWorld, with the three metrics improving simultaneously rather than trading off. The experiment changes nothing about the learning algorithm, the model, or the task set; the only difference is how the trace is presented. That this single substitution moves all three metrics in the same direction suggests that message format is not an incidental implementation

choice but a load-bearing component of context learning infrastructure. The current evaluation is limited to AppWorld; extending to other agent benchmarks—SWE-bench [17], TAU-bench [53], WebArena [61], AgentBench [25]—and to additional trajectory-learning methods remains an open direction.

References

- [1] A. AlSayyad, K. Y. Huang, and R. Pal. Agenttrace: A structured logging framework for agent system observability, 2026. URL <https://arxiv.org/abs/2602.10133>.
- [2] S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal. Agentrx: Diagnosing ai agent failures from execution trajectories, 2026. URL <https://arxiv.org/abs/2602.02475>.
- [3] M. Chen, Y. Li, Y. Yang, S. Yu, B. Lin, and X. He. Automanual: Constructing instruction manuals by llm agents via interactive environmental learning, 2024. URL <https://arxiv.org/abs/2405.16247>.
- [4] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- [5] Cognition. Cognition. Cognition. URL <https://cognition.ai/blog/agent-trace>.
- [6] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian. Trail: Trace reasoning and agentic issue localization, 2025. URL <https://arxiv.org/abs/2505.08638>.
- [7] Y. Du, M. Tian, S. Ronanki, S. Rongali, S. Bodapati, A. Galstyan, A. Wells, R. Schwartz, E. A. Huerta, and H. Peng. Context length alone hurts llm performance despite perfect retrieval, 2025. URL <https://arxiv.org/abs/2510.05381>.
- [8] G. Fang, V. Isahagian, K. R. Jayaram, R. Kumar, V. Muthusamy, P. Oum, and G. Thomas. Trajectory-informed memory generation for self-improving agent systems, 2026. URL <https://arxiv.org/abs/2603.10600>.
- [9] L. Feng, F. Yang, F. Chen, X. Cheng, H. Xu, Z. Wan, M. Yan, and B. An. Agentocr: Reimagining agent history via optical self-compression, 2026. URL <https://arxiv.org/abs/2601.04786>.
- [10] S. Forouzandeh, W. Peng, P. Moradi, X. Yu, and M. Jalili. Learning hierarchical procedural memory for llm agents through bayesian selection and contrastive refinement, 2025. URL <https://arxiv.org/abs/2512.18950>.
- [11] Y. Fu, D.-K. Kim, J. Kim, S. Sohn, L. Logeswaran, K. Bae, and H. Lee. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents, 2024. URL <https://arxiv.org/abs/2403.08978>.
- [12] J. He, M. Rungta, D. Koleczek, A. Sekhon, F. X. Wang, and S. Hasan. Does prompt formatting have any impact on llm performance?, 2024. URL <https://arxiv.org/abs/2411.10541>.
- [13] M. Y. Hu, B. Van Durme, J. Andreas, and H. Jhamtani. Sample-efficient online learning in llm agents via hindsight trajectory rewriting, 2025. URL <https://arxiv.org/abs/2510.10304>.
- [14] Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, S. Jin, J. Tan, Y. Yin, J. Liu, Z. Zhang, Z. Sun, Y. Zhu, H. Sun, B. Peng, Z. Cheng, X. Fan, J. Guo, X. Yu, Z. Zhou, Z. Hu, J. Huo, J. Wang, Y. Niu, Y. Wang, Z. Yin, X. Hu, Y. Liao, Q. Li, K. Wang, W. Zhou, Y. Liu, D. Cheng, Q. Zhang, T. Gui, S. Pan, Y. Zhang, P. Torr, Z. Dou, J.-R. Wen, X. Huang, Y.-G. Jiang, and S. Yan. Memory in the age of ai agents, 2025. URL <https://arxiv.org/abs/2512.13564>.
- [15] R. Hutter and M. Pradel. Agentstepper: Interactive debugging of software development agents, 2026. URL <https://arxiv.org/abs/2602.06593>.
- [16] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. Llmilingua: Compressing prompts for accelerated inference of large language models, 2023. URL <https://arxiv.org/abs/2310.05736>.

- [17] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [18] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan. Acon: Optimizing context compression for long-horizon llm agents, 2025. URL <https://arxiv.org/abs/2510.00615>.
- [19] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023. URL <https://arxiv.org/abs/2310.03714>.
- [20] A. B. Labate, V. M. de Sousa, S. R. Fiorini, L. G. Azevedo, R. M. Thiago, and V. T. da Silva. Solving context window overflow in ai agents, 2025. URL <https://arxiv.org/abs/2511.22729>.
- [21] langchain-ai. langsmith-sdk. <https://github.com/langchain-ai/langsmith-sdk>, 2023. URL <https://github.com/langchain-ai/langsmith-sdk>. LangSmith Client SDK Implementations.
- [22] langfuse. langfuse. <https://github.com/langfuse/langfuse>, 2023. URL <https://github.com/langfuse/langfuse>. Open source LLM engineering platform: LLM Observability, metrics, evals, prompt management, playground, datasets. Integrates with OpenTelemetry, Langchain, OpenAI SDK, LiteLLM, and more. YC W23.
- [23] T. Lindenbauer, I. Slinko, L. Felder, E. Bogomolov, and Y. Zharov. The complexity trap: Simple observation masking is as efficient as llm summarization for agent context management, 2025. URL <https://arxiv.org/abs/2508.21433>.
- [24] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts, 2023. URL <https://arxiv.org/abs/2307.03172>.
- [25] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. Agentbench: Evaluating llms as agents, 2023. URL <https://arxiv.org/abs/2308.03688>.
- [26] Y. Liu, J. Xu, L. L. Zhang, Q. Chen, X. Feng, Y. Chen, Z. Guo, Y. Yang, and P. Cheng. Beyond prompt content: Enhancing llm performance via content-format integrated prompt optimization, 2025. URL <https://arxiv.org/abs/2502.04295>.
- [27] M. Ma, J. Zhang, F. Yang, Y. Kang, Q. Lin, S. Rajmohan, and D. Zhang. Dover: Intervention-driven auto debugging for llm multi-agent systems, 2025. URL <https://arxiv.org/abs/2512.06749>.
- [28] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu. A survey of context engineering for large language models, 2025. URL <https://arxiv.org/abs/2507.13334>.
- [29] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. Memgpt: Towards llms as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- [30] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL <https://arxiv.org/abs/2304.03442>.
- [31] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval, 2024. URL <https://arxiv.org/abs/2401.18059>.

- [32] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>.
- [33] J. She. Agentrm: An os-inspired resource manager for llm agent systems, 2026. URL <https://arxiv.org/abs/2603.13110>.
- [34] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [35] Y. Song, D. Yin, X. Yue, J. Huang, S. Li, and B. Y. Lin. Trial and error: Exploration-based trajectory optimization for llm agents, 2024. URL <https://arxiv.org/abs/2403.02502>.
- [36] Y. Sui, M. Zhou, M. Zhou, S. Han, and D. Zhang. Table meets llm: Can large language models understand structured table data? a benchmark and empirical study, 2023. URL <https://arxiv.org/abs/2305.13062>.
- [37] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths. Cognitive architectures for language agents, 2023. URL <https://arxiv.org/abs/2309.02427>.
- [38] W. Sun, M. Lu, Z. Ling, K. Liu, X. Yao, Y. Yang, and J. Chen. Scaling long-horizon llm agent via context-folding, 2025. URL <https://arxiv.org/abs/2510.11967>.
- [39] M. Suzgun, M. Yuksekgonul, F. Bianchi, D. Jurafsky, and J. Zou. Dynamic cheatsheet: Test-time learning with adaptive memory, 2025. URL <https://arxiv.org/abs/2504.07952>.
- [40] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models, 2024. URL <https://arxiv.org/abs/2408.02442>.
- [41] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents, 2024. URL <https://arxiv.org/abs/2407.18901>.
- [42] Trychroma. Context rot: How increasing input tokens impacts llm performance. URL <https://research.trychroma.com/context-rot>.
- [43] N. Verma. Active context compression: Autonomous memory management in llm agents, 2026. URL <https://arxiv.org/abs/2601.07190>.
- [44] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig. Openhands: An open platform for ai software developers as generalist agents, 2024. URL <https://arxiv.org/abs/2407.16741>.
- [45] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory, 2024. URL <https://arxiv.org/abs/2409.07429>.
- [46] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2022. URL <https://arxiv.org/abs/2201.11903>.
- [47] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [48] Y.-A. Xiao, P. Gao, C. Peng, and Y. Xiong. Reducing cost of llm agents with trajectory reduction, 2025. URL <https://arxiv.org/abs/2509.23586>.
- [49] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-mem: Agentic memory for llm agents, 2025. URL <https://arxiv.org/abs/2502.12110>.

- [50] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [51] S.-X. Yang, C. Wang, H. Zhang, W. Yu, L. Wu, J. Gui, D. Yang, Y. Cen, Z. Feng, B. Wen, Y. Wang, L. Zhong, J. Ren, L. Zhang, and J. Tang. Tracesir: A multi-agent framework for structured analysis and reporting of agentic execution traces, 2026. URL <https://arxiv.org/abs/2603.00623>.
- [52] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models, 2022. URL <https://arxiv.org/abs/2210.03629>.
- [53] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL <https://arxiv.org/abs/2406.12045>.
- [54] R. Ye, Z. Zhang, K. Li, H. Yin, Z. Tao, Y. Zhao, L. Su, L. Zhang, Z. Qiao, X. Wang, P. Xie, F. Huang, S. Chen, J. Zhou, and Y. Jiang. Agentfold: Long-horizon web agents with proactive context management, 2025. URL <https://arxiv.org/abs/2510.24699>.
- [55] Z. Yuan, S. Yuan, and L. Xie. Rpms: Enhancing llm-based embodied planning through rule-augmented memory synergy, 2026. URL <https://arxiv.org/abs/2603.17831>.
- [56] W. Zeng, Y. Huang, and J. He. Loca-bench: Benchmarking language agents under controllable and extreme context growth, 2026. URL <https://arxiv.org/abs/2602.07962>.
- [57] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. Yan. Agentracer: Who is inducing failure in the llm agentic systems?, 2025. URL <https://arxiv.org/abs/2509.03312>.
- [58] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun. Agentic context engineering: Evolving contexts for self-improving language models, 2025. URL <https://arxiv.org/abs/2510.04618>.
- [59] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang. Expel: Llm agents are experiential learners, 2023. URL <https://arxiv.org/abs/2308.10144>.
- [60] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. Sglang: Efficient execution of structured language model programs, 2023. URL <https://arxiv.org/abs/2312.07104>.
- [61] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. Webarena: A realistic web environment for building autonomous agents, 2023. URL <https://arxiv.org/abs/2307.13854>.